

Training Course Design & Student Evaluation Multi-Agent System

Designed by: Mohammad Abu Alqassem

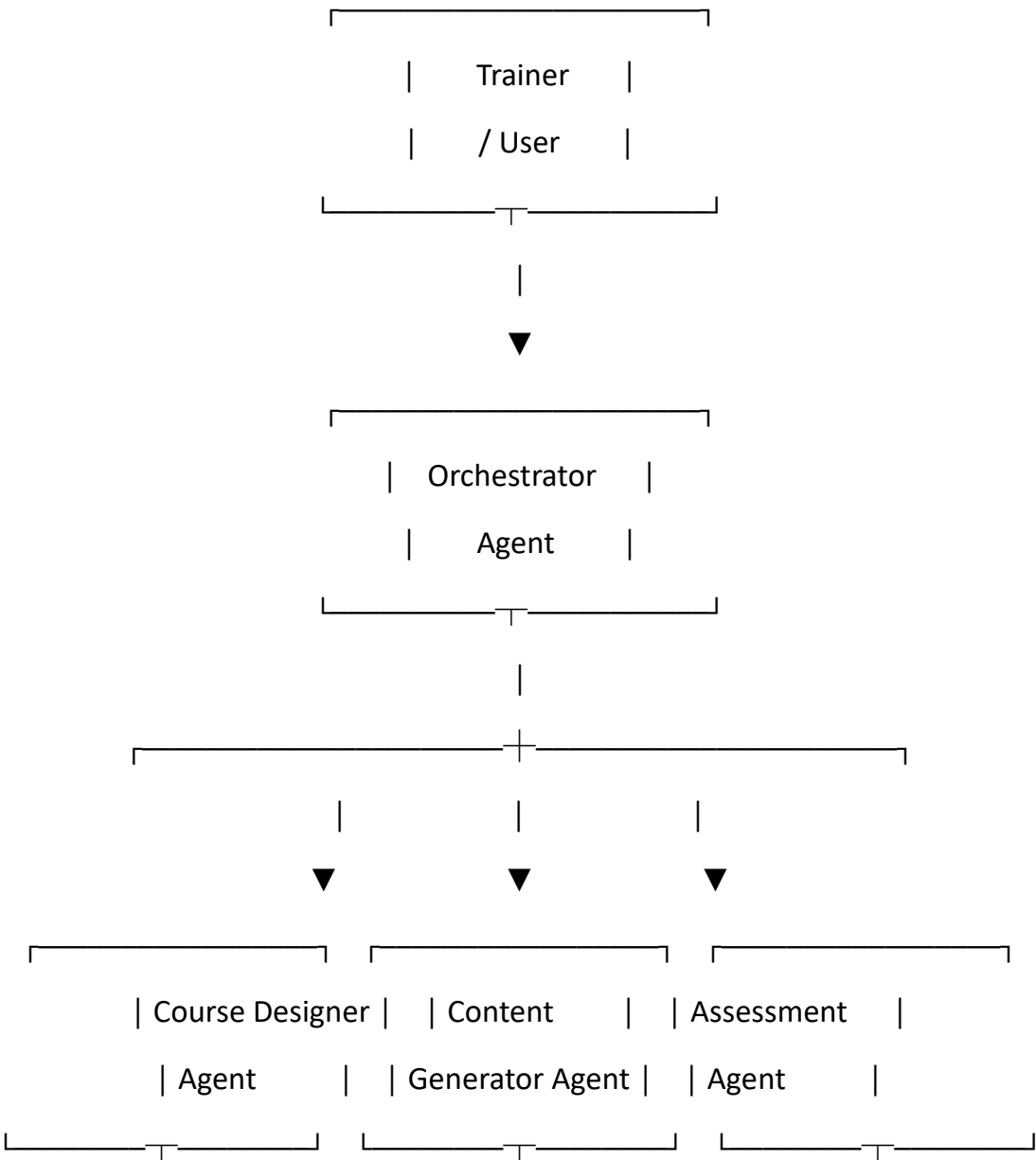
Supervised by: Dr. Fakhr El-Din Saeed

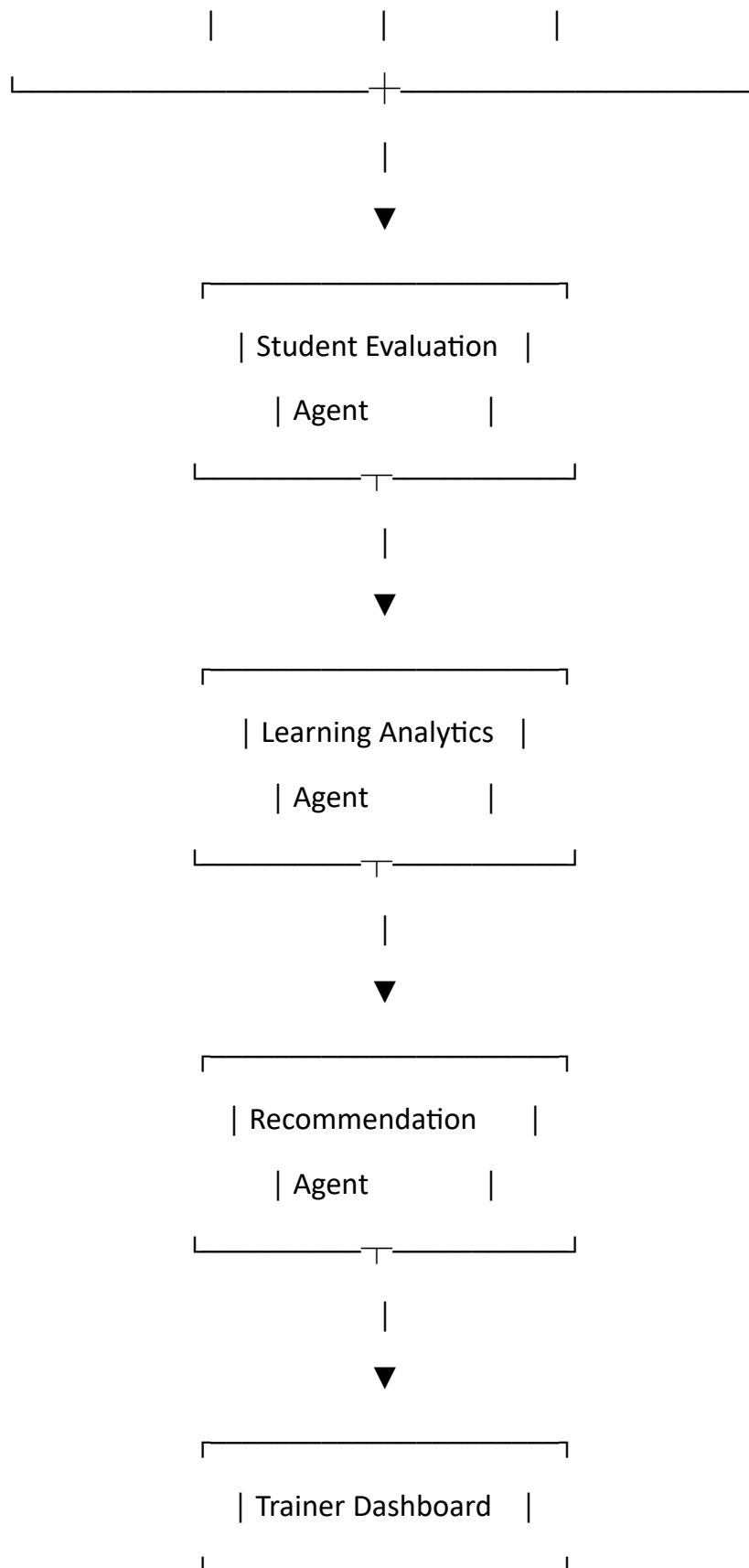
المرحلة الأولى من المشروع :

نبدأ الآن التصميم الرسمي للمشروع قبل كتابة أي كود. وهذه خطوة مهمة جدًا لأننا سنبنّي الكود لاحقًا انطلاقًا من هذه المواصفات، وليس العكس.

1. System Architecture

سنستخدم بنية **Hierarchical Multi-Agent Architecture**، بحيث يوجد Agent رئيسي يسمى **Orchestrator Agent**، وهو المسؤول عن إدارة سير العمل وتحديد الـ Agent الذي يجب أن يعمل في كل مرحلة.





#	Agent	المسؤولية
1	Orchestrator Agent	إدارة workflow
2	Course Designer Agent	تصميم الدورة
3	Content Generator Agent	إنشاء المحتوى والأنشطة
4	Assessment Agent	إنشاء الاختبارات
5	Student Evaluation Agent	تقييم إجابات الطلاب
6	Learning Analytics Agent	تحليل أداء الطلاب
7	Recommendation Agent	تقديم التوصيات

هذه البنية تسمح لنا لاحقًا بإضافة Agents أخرى دون إعادة تصميم النظام بالكامل.

3. Agent 1 — Orchestrator Agent

هذا هو العقل المنظم للنظام.

: Goal

تحديد المهمة الحالية، اختيار الـ Agent المناسب، التحقق من نتائج كل Agent ، وإدارة الانتقال بين المراحل.

: Input : مثلاً:

Create a 30-hour Python course for beginners.

أو:

Analyze the performance of student S001.

: Output : لا نريد منه إجابة نصية فقط.

نريد قرارًا منظمًا:

NEXT_AGENT

TASK

REASON

REQUIRED_INPUT

Pydantic Schema

```
class AgentTask(BaseModel):
```

```
    next_agent: str
```

```
    task: str
```

```
    reason: str
```

```
    required_input: list[str]
```

Tools : في البداية:

course_state_tool

student_state_tool

assessment_state_tool

لاحقًا يمكن إضافة:

database_tool

report_tool

notification_tool

System Prompt

المبدأ الأساسي للـ Prompt:

You are the Orchestrator Agent.

Your responsibility is to manage the training workflow.

You must select the appropriate agent based on the current state and task.

Never invent missing data.

Never perform another agent's specialized task.

Always return a valid structured decision.

Gates : يجب أن يتحقق النظام من:

✓ Valid agent name

✓ Valid task

✓ Required state exists

✓ No unauthorized action

✓ Valid Pydantic output

Invalid agent selected

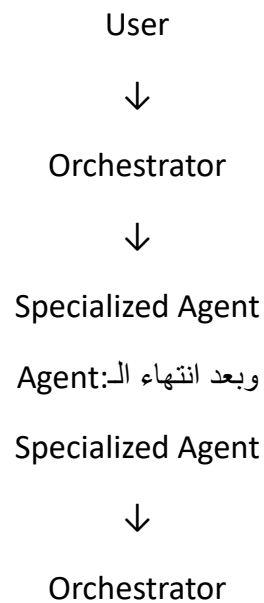
Missing course state

Invalid task

Malformed structured output

Repeated routing

: Handoff



نقيس : Evaluation Criteria

Routing Accuracy

Schema Validity

No Invalid Handoffs

No Infinite Loops

Correct Workflow Completion

4. Agent 2 — Course Designer Agent :

Goal : إنشاء خطة دورة تدريبية كاملة بناءً على متطلبات المدرب.

Input :

Course title

Target audience

Level

Duration

Number of students

Prerequisites

Training goals

مثلاً:

Course: Python Programming

Level: Beginner

Duration: 30 hours

Students: 20

Output:

Course Description

Learning Objectives

Modules

Topics

Duration

Prerequisites

Assessment Strategy

Pydantic Schema :

سنبدأ بهذا:

```
class LearningObjective(BaseModel):
```

```
    id: str
```

```
    description: str
```

```
    level: str
```

```
class Module(BaseModel):
```

```
    id: str
```

```
    title: str
```

```
    duration_hours: float
```

```
    objectives: list[LearningObjective]
```

```
class CoursePlan(BaseModel):
```

```
    title: str
```

```
    description: str
```

```
    level: str
```

```
    duration_hours: float
```

```
    prerequisites: list[str]
```

```
    modules: list[Module]
```

Tools :

Course Validation Tool

Duration Calculator

Course Database

Gates :

✓ Duration must be positive

✓ Module durations must sum to course duration

✓ At least one learning objective

✓ Every module must have objectives

✓ No duplicate module IDs

مثلاً:

Course Duration = 30 hours

Module 1 = 5

Module 2 = 5

Module 3 = 4

Module 4 = 6

Module 5 = 5

Module 6 = 5

Total = 30 ✓

Failure Conditions :

Missing course duration

No learning objectives

Invalid duration

Duplicate modules

Module/objective mismatch

Handoff :

Course Designer



Content Generator

بعد إنشاء CoursePlan والتحقق منه.

Evaluation:

Objective Quality

Module Coherence

Time Allocation

Prerequisite Validity

Objective Coverage

Schema Validity

5. Agent 3 — Content Generator Agent:

Goal : إنشاء المحتوى التعليمي والأنشطة لكل Module.

Input:

CoursePlan

Module

Learning Objectives

Output:

Lessons

Examples

Exercises

Practical Tasks

Learning Resources

Schema :

```
class LearningActivity(BaseModel):
```

```
    id: str
```

```
    title: str
```

```
    description: str
```

```
    activity_type: str
```

```
class Lesson(BaseModel):
```

```
    id: str
```

```
    title: str
```

```
    objectives: list[str]
```

```
    activities: list[LearningActivity]
```

```
class ModuleContent(BaseModel):
```

```
    module_id: str
```

```
    lessons: list[Lesson]
```

Tools :

Content Repository

Document Reader

Exercise Generator

Gates:

✓ Every lesson has objectives

✓ Objectives belong to module

✓ Activities are linked to lesson

✓ No empty lesson

Failure Conditions :

Unknown objective

Empty lesson

Duplicate content

Content unrelated to objective

Handoff :

Content Generator



Assessment Agent

6. Agent 4 — Assessment Agent :

Goal : إنشاء Assessment مرتبط مباشرة بالـ Learning Objectives.

Input :

CoursePlan

LearningObjectives

ModuleContent

Output :

Questions

Answers

Difficulty

Objective Mapping

Assessment Metadata

مثلاً:

Q001

Objective: LO-03

Difficulty: Easy

Type: MCQ

Schema :

```
class Question(BaseModel):
```

```
    id: str
```

```
    question: str
```

```
    question_type: str
```

difficulty: str
objective_id: str
options: list[str]
correct_answer: str

```
class Assessment(BaseModel):
```

```
    id: str  
    title: str  
    questions: list[Question]  
    total_marks: float
```

Gates :

- ✓ Every question has an objective
- ✓ Correct answer exists
- ✓ MCQ has valid options
- ✓ No duplicate question IDs
- ✓ Difficulty is valid

Failure Conditions :

Missing answer
Invalid objective
Duplicate question
Invalid difficulty
Question unrelated to objective

Handoff : بعد اعتماد الاختبار:

Assessment



Student Evaluation

7. Agent 5 — Student Evaluation Agent :

Goal : تقييم إجابات الطالب وإنتاج Student Performance Profile.

Input :

Student

Assessment

Student Answers

Output: مثلاً:

Student: S001

Overall Score: 68%

Strong:

Variables

Conditions

Weak:

Loops

Functions

Schema :

```
class ObjectiveResult(BaseModel):
```

```
    objective_id: str
```

```
    score: float
```

```
    mastery_level: str
```

```
class StudentEvaluation(BaseModel):
```

```
    student_id: str
```

```
    assessment_id: str
```

```
    overall_score: float
```

```
    objective_results: list[ObjectiveResult]
```

Tools : هنا يجب أن نستخدم Tool للحساب.

```
calculate_score()
```

بدل أن نترك الـ LLM يقوم بالحساب.

Gates :

- ✓ Student exists
- ✓ Assessment exists
- ✓ Answers correspond to questions
- ✓ Score calculated by tool
- ✓ Score range = 0–100

Failure Conditions :

- Unknown student
- Unknown question
- Missing answer
- Invalid score
- Assessment mismatch

Handoff :

Student Evaluation



Learning Analytics

8. Agent 6 — Learning Analytics Agent:

هذا الـ Agent يحول النتائج إلى **Learning Insights**.

Goal : اكتشاف نقاط القوة والضعف لدى الطلاب.

Input:

- Student Evaluations
- Class Results
- Learning Objectives

Output:

- Class Performance
- Objective Mastery
- Weak Areas
- Strong Areas

Learning Gaps

مثلاً:

LO-01 → 88%

LO-02 → 81%

LO-03 → 56%

LO-04 → 48%

فيستنتج:

Learning Gap:

LO-04

Schema

```
class LearningGap(BaseModel):
```

```
    objective_id: str
```

```
    mastery_score: float
```

```
    severity: str
```

```
class LearningAnalysis(BaseModel):
```

```
    class_average: float
```

```
    strong_objectives: list[str]
```

```
    weak_objectives: list[str]
```

```
    learning_gaps: list[LearningGap]
```

Gates :

✓ Valid scores

✓ Objectives exist

✓ Calculations consistent

✓ Thresholds applied consistently

Failure Conditions :

No evaluation data

Invalid score

Unknown objective

Contradictory analysis

Handoff :

Learning Analytics



Recommendation Agent

9. Agent 7 — Recommendation Agent :

Goal : تحويل التحليل إلى إجراءات تعليمية.

Input :

Student Evaluation

Learning Analysis

Course Content

Output :

مثلاً:

Recommendation:

1. Reteach Functions.
2. Provide 5 additional exercises.
3. Give practical task.
4. Conduct retest.

Schema :

```
class Recommendation(BaseModel):
```

```
    id: str
```

```
    target: str
```

```
    issue: str
```

```
    action: str
```

```
    priority: str
```

```
    rationale: str
```

```
class RecommendationReport(BaseModel):
```

```
    recommendations: list[Recommendation]
```

Gates :

- ✓ Recommendation based on evidence
- ✓ Target exists
- ✓ Action is relevant
- ✓ Priority is valid

Failure Conditions :

- Unsupported recommendation
- No evidence
- Unknown student
- Unknown objective

Handoff :

Recommendation Agent



Trainer

10- الـ Workflow الكامل : الآن نستطيع تعريف الـ workflow الرسمي:

USER



ORCHESTRATOR



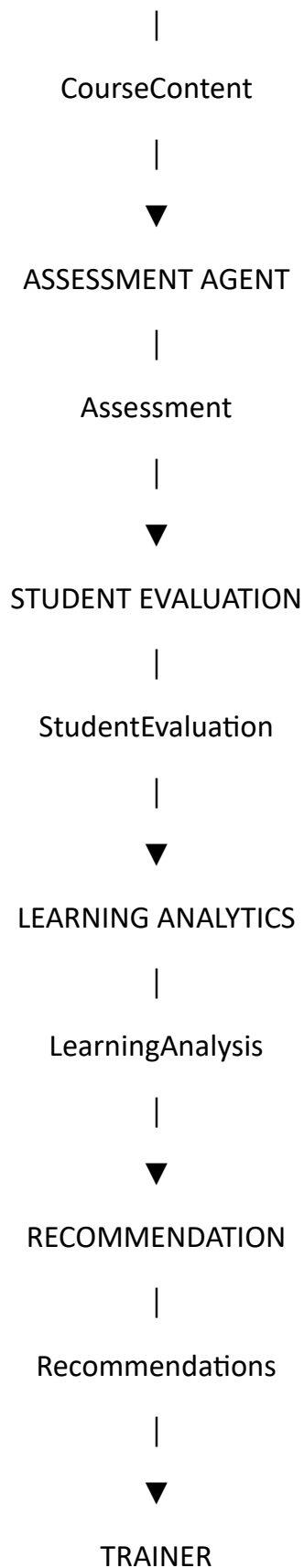
COURSE DESIGNER



CoursePlan



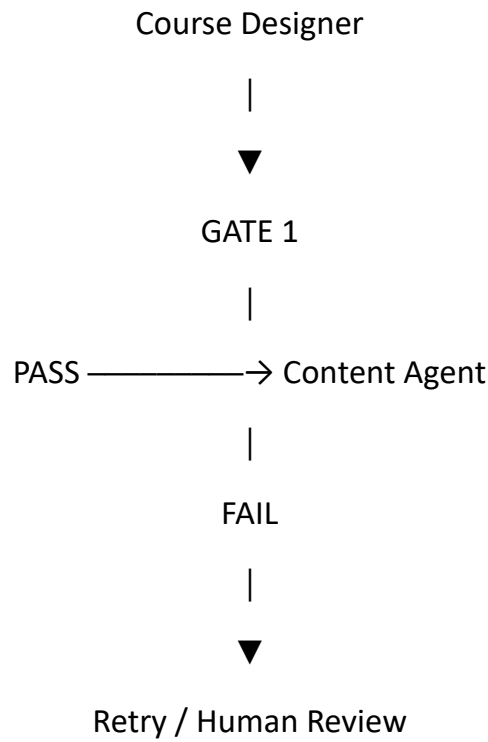
CONTENT GENERATOR



11. أهم مبدأ في التصميم Gates : بما أننا تعمل على **Agentic AI**، لن نجعل الـ Agents يمررون النتائج بشكل أعمى.

كل مرحلة لها Gate.

مثلاً:



مثال:

إذا أنتج: Course Designer:

Course Duration = 30 hours

Modules:

5 + 5 + 5 = 15 hours

فاله Gate يرفض النتيجة:

✗ COURSE_VALIDATION_FAILED

Expected: 30 hours

Actual: 15 hours

ولا يسمح للنظام بالانتقال إلى Content Agent.

وهذا بالضبط النوع من **guardrails** الذي يجعل النظام Agentic System منضبطاً، وليس مجرد سلسلة من استدعاءات LLM.

12. Failure Handling

سنستخدم نمطاً موحداً:

```
class GateResult(BaseModel):
```

```
    passed: bool
```

```
    gate_name: str
```

```
errors: list[str]
```

لا يستطيع الانتقال للمرحلة التالية إلا Agent والـ

```
if gate_result.passed:
```

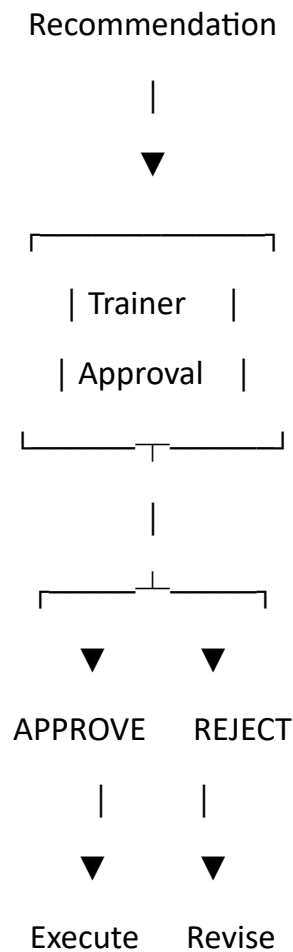
```
    handoff()
```

```
else:
```

```
    handle_failure()
```

13. Human-in-the-Loop :

سنضع نقطة تدخل للمدرب في القرارات المهمة:



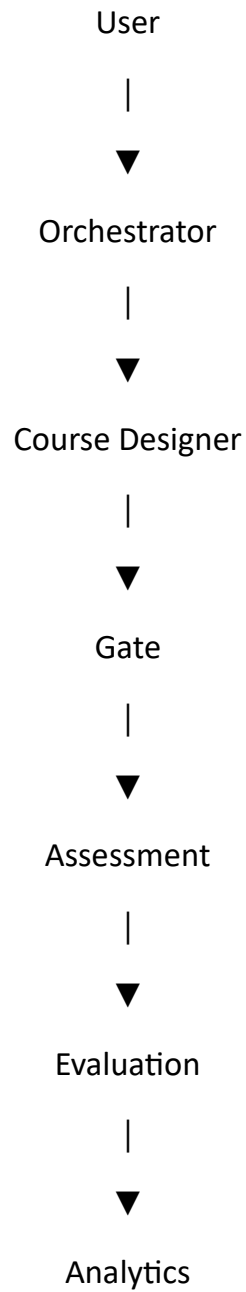
مثلاً:

إعادة تصميم Module بسبب ضعف الطلاب.

النظام يقترح، والمدرب يعتمد.

14- أول نسخة MVP

لن نقوم ببناء الـ 7 Agents من البداية.



ثم نضيف:

Content Generator + Recommendation Agent

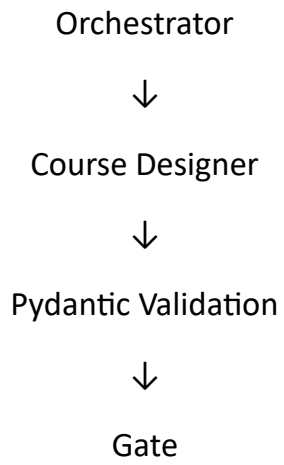
بعد التأكد أن الأساس يعمل.

15- أول هدف برمجي لنا

Input :

Create a 30-hour beginner Python course.

Processing



Output :

CoursePlan

مثل:

Course: Python Programming

Duration: 30 hours

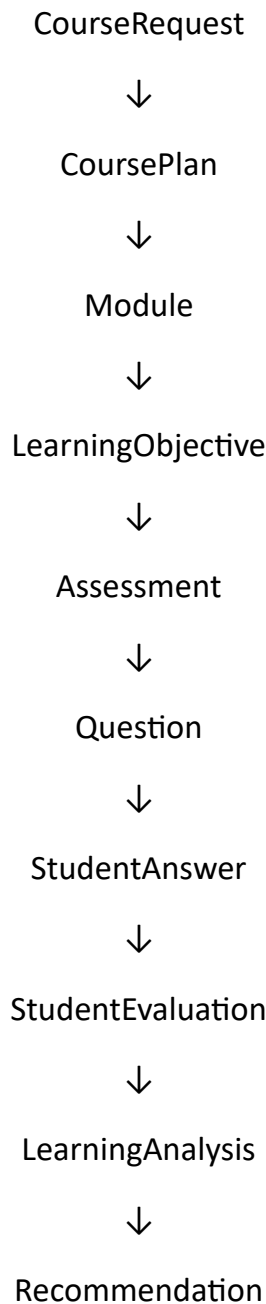
Modules:

1. Python Basics
2. Variables and Data Types
3. Conditions
4. Loops
5. Functions
6. Data Structures
7. Final Project

إذا نجح هذا الجزء، ننتقل إلى **Content Agent**.

System Architecture + Agent Specifications.

الخطوة العملية التالية التي سننفذها هي تصميم الـ **Pydantic Schemas** بالكامل قبل كتابة الـ **Agents**، لأن الـ Schema سيكون "العقد" الذي يربط جميع أجزاء النظام:



وبعدها سنكتب schemas/ بالكامل ثم نبدأ بأول **Course Designer Agent** :

Agent ، مع **System Prompt + Gate + Failure Handling + Trace**.